

PYTHON VISUALIZATION LIBRARIES

A Comprehensive Guide to Matplotlib and Seaborn

Internship Task – Visualization Library Documentation

Selected Libraries:

- Matplotlib
- Seaborn

Prepared By:

Sulthan Vaishnavi

Course :

B.Tech – Computer Science (Data Science)

College:

Institute of Aeronautical Engineering

Task Level :

Beginner

Submission Date:

15/09/2026

TABLE OF CONTENTS

1. Introduction to Data Visualization
2. Matplotlib Overview
3. Installing and Importing Matplotlib
4. Matplotlib Graph Types
 - Line Plot
 - Bar Chart
 - Scatter Plot
 - Histogram
 - Pie Chart
 - Box Plot
5. Seaborn Overview
6. Installing and Importing Seaborn
7. Seaborn Dataset Example
8. Seaborn Graph Types
 - Scatter Plot
 - Bar Plot
 - Histogram
 - Box Plot
 - Violin Plot
 - Heatmap
9. Matplotlib vs Seaborn Comparison
10. Choosing the Right Graph
11. Practical Mini Project
12. Strengths and Weaknesses
13. Large Dataset Considerations
14. Visualization Best Practices
15. Conclusion
16. Resources

1. Introduction to Data Visualization

1.1 What is Data Visualization?

Data visualization is the process of representing data in graphical or visual form, such as charts, graphs, plots, and diagrams.

Instead of examining large tables of numerical values, visualization allows users to understand patterns, trends, relationships, distributions, and comparisons more easily.

For example, a line chart can show how sales change over several months, while a scatter plot can show the relationship between two variables.

1.2 Importance of Data Visualization

Data visualization is important because it:

- Makes complex data easier to understand.
- Helps identify trends and patterns.
- Makes comparisons easier.
- Helps detect unusual values and outliers.
- Supports data-driven decision making.
- Communicates analytical results effectively.
- Makes reports and dashboards more informative.

1.3 Python Visualization Libraries

Python provides several libraries for creating visualizations. Some commonly used libraries are:

- Matplotlib
- Seaborn
- Plotly
- Bokeh
- Pandas

This documentation focuses on **Matplotlib and Seaborn**.

Matplotlib provides a flexible foundation for creating many types of static graphs, while Seaborn provides a higher-level interface particularly useful for statistical data visualization.

2. Matplotlib Overview

2.1 Introduction

Matplotlib is a widely used Python library for creating static, animated, and interactive visualizations.

It provides detailed control over elements such as:

- Titles
- Axis labels
- Legends
- Colors
- Markers
- Line styles
- Figure size
- Grid lines
- Annotations

Matplotlib is commonly used in data analysis, scientific computing, engineering, research, education, and reporting.

The most commonly used module is `matplotlib.pyplot`.

2.2 Key Features

Important features of Matplotlib include:

1. Supports many types of graphs.
2. Provides extensive customization.
3. Works well with NumPy and Pandas.
4. Allows control over almost every part of a chart.
5. Suitable for scientific and technical visualization.
6. Can create publication-quality figures.
7. Supports multiple plots within a single figure.

2.3 Typical Use Cases

Matplotlib can be used for:

- Sales analysis
- Scientific graphs
- Mathematical functions
- Statistical analysis
- Engineering data
- Time-series visualization
- Academic research

3. Installing and Importing Matplotlib

3.1 Installation

Matplotlib can be installed using pip:

```
pip install matplotlib
```

3.2 Importing Matplotlib

```
import matplotlib.pyplot as plt
```

`pyplot` provides functions for creating and modifying plots.

4. Matplotlib Graph Types

4.1 Line Plot

Description

A line plot displays data points connected by straight lines. It is especially useful for showing changes and trends over a continuous variable such as time.

Common Use Cases

- Monthly sales trends
- Temperature changes
- Stock price trends
- Website traffic
- Performance over time

Code Example

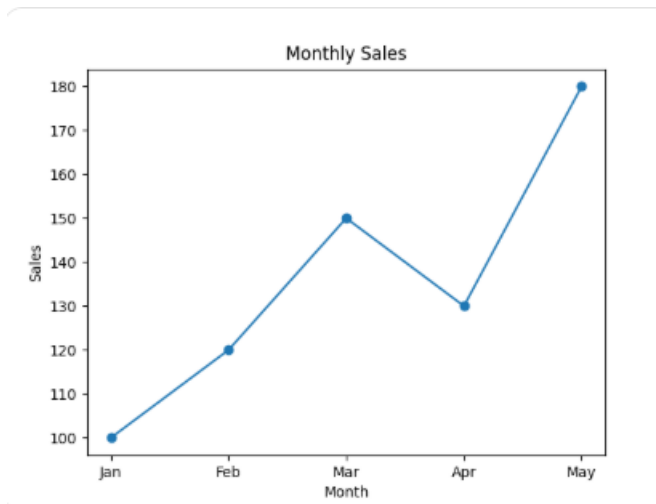
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [100, 120, 150, 130, 180]

plt.plot(months, sales, marker="o")

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```



Explanation

- `plt.plot()` creates the line plot.
- `marker="o"` displays markers at data points.
- `plt.title()` adds a title.
- `plt.xlabel()` labels the x-axis.
- `plt.ylabel()` labels the y-axis.
- `plt.show()` displays the graph.

4.2 Bar Chart

Description

A bar chart represents categorical data using rectangular bars. The height or length of each bar represents the corresponding value.

Common Use Cases

- Comparing product sales
- Comparing student marks
- Comparing company revenue
- Comparing categories

Code Example

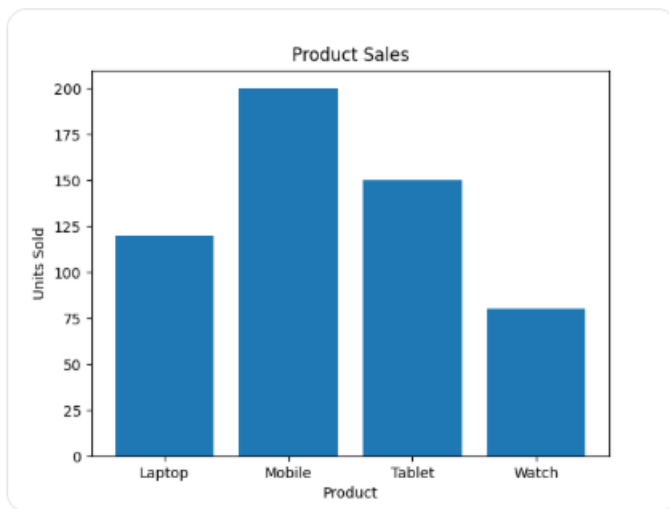
```
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet", "Watch"]
sales = [120, 200, 150, 80]

plt.bar(products, sales)

plt.title("Product Sales")
plt.xlabel("Product")
plt.ylabel("Units Sold")
```

```
plt.show()
```



Explanation

The x-axis contains product categories, while the y-axis represents the number of units sold.

4.3 Scatter Plot

Description

A scatter plot displays individual data points to show the relationship between two numerical variables.

Common Use Cases

- Study hours vs marks
- Height vs weight
- Advertising expenditure vs sales
- Temperature vs electricity consumption

Code Example

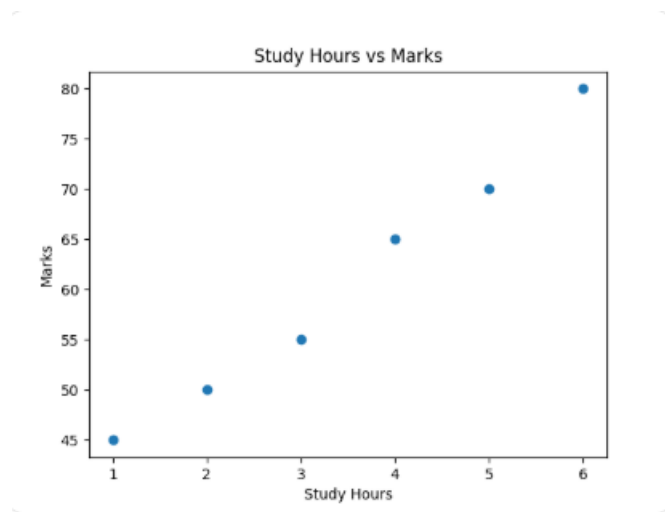
```
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5, 6]
marks = [45, 50, 55, 65, 70, 80]

plt.scatter(hours, marks)

plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
```

```
plt.show()
```



Interpretation

The graph can help determine whether an apparent relationship exists between study hours and marks.

4.4 Histogram

Description

A histogram displays the distribution of numerical data by grouping values into intervals called bins.

Common Use Cases

- Exam mark distribution
- Age distribution
- Salary distribution
- Measurement analysis

Code Example

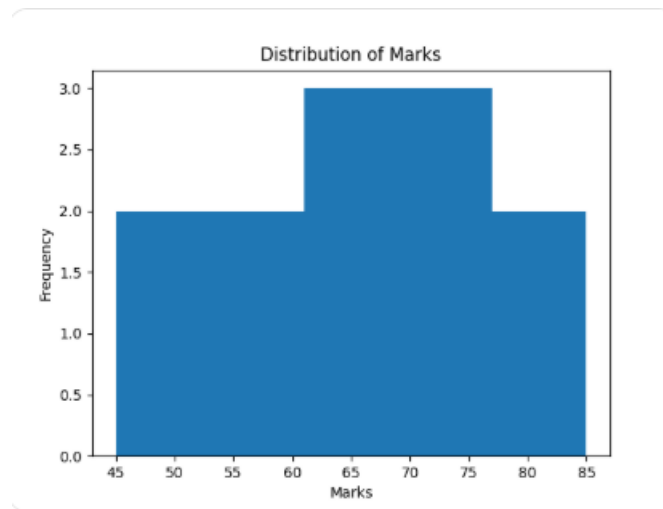
```
import matplotlib.pyplot as plt

marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 80, 85]

plt.hist(marks, bins=5)

plt.title("Distribution of Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
```

```
plt.show()
```



Explanation

The `bins` parameter determines how the numerical values are grouped.

4.5 Pie Chart

Description

A pie chart represents parts of a whole using circular sections. Each section represents a category's proportion of the total.

Common Use Cases

- Expense distribution
- Market share
- Budget allocation
- Category proportions

Code Example

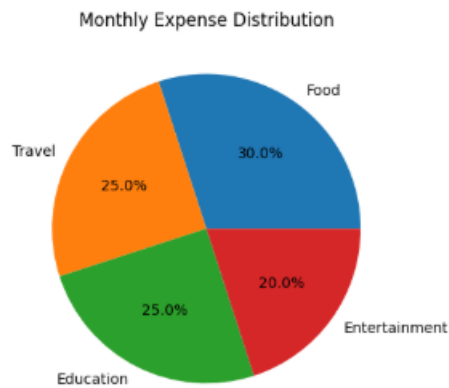
```
import matplotlib.pyplot as plt

categories = ["Food", "Travel", "Education", "Entertainment"]
expenses = [30, 25, 25, 20]

plt.pie(
    expenses,
    labels=categories,
    autopct="%1.1f%%"
)

plt.title("Monthly Expense Distribution")
```

```
plt.show()
```



Explanation

`autopct` displays the percentage of each category.

4.6 Box Plot

Description

A box plot summarizes the distribution of numerical data using statistical measures such as the median, quartiles, and potential outliers.

Common Use Cases

- Salary analysis
- Exam marks
- Detecting outliers
- Comparing distributions

Code Example

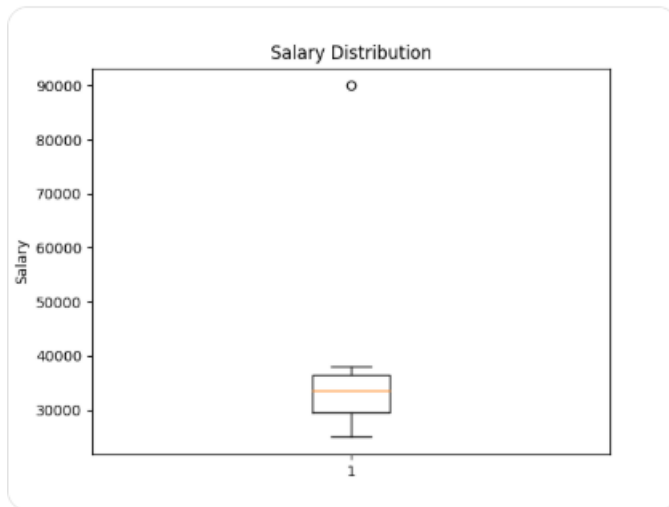
```
import matplotlib.pyplot as plt

salary = [25000, 28000, 30000, 32000, 35000, 36000, 38000, 90000]

plt.boxplot(salary)

plt.title("Salary Distribution")
plt.ylabel("Salary")
```

```
plt.show()
```



Interpretation

The box plot helps identify the central distribution of values and potential unusual observations.

5. Seaborn Overview

5.1 Introduction

Seaborn is a Python data visualization library built on top of Matplotlib.

It provides a high-level interface for creating attractive and informative statistical graphics.

Seaborn is particularly useful when working with structured datasets and Pandas DataFrames.

5.2 Key Features

Seaborn provides:

- Statistical visualization.
- Attractive default styles.
- Easy integration with Pandas DataFrames.
- Convenient functions for categorical data.
- Distribution plots.
- Relationship plots.
- Correlation heatmaps.
- Easy grouping using categorical variables.
- Integration with Matplotlib for further customization.

5.3 Typical Use Cases

Seaborn is commonly used for:

- Exploratory Data Analysis (EDA)
- Statistical analysis
- Correlation analysis
- Distribution analysis
- Categorical data analysis

6. Installing and Importing Seaborn

Installation

```
pip install seaborn
```

Importing

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Seaborn is normally imported using the alias `sns`.

7. Seaborn Dataset Example

Seaborn provides example datasets that can be used for learning and demonstration.

One commonly used dataset is the **tips dataset**, which contains information about restaurant bills and tips.

Code

```
import seaborn as sns

data = sns.load_dataset("tips")

print(data.head())
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

The dataset contains variables such as:

- total_bill
- tip
- sex

- smoker
- day
- time
- size

This dataset is useful for demonstrating relationships, distributions, categorical comparisons, and correlation.

8. Seaborn Graph Types

8.1 Scatter Plot

Description

A Seaborn scatter plot displays individual observations and helps analyze the relationship between two numerical variables.

Use Case

For example, we can examine the relationship between total restaurant bill and tip amount.

Code

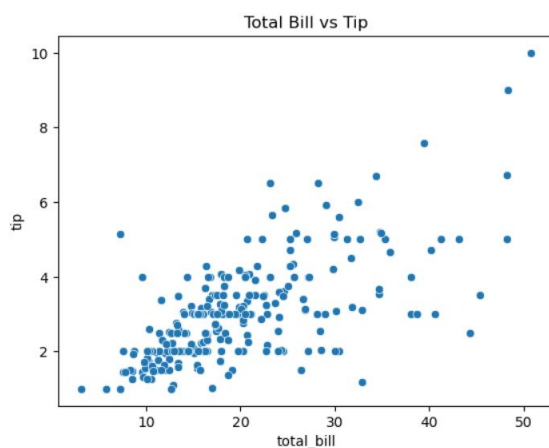
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")

sns.scatterplot(
    data=data,
    x="total_bill",
    y="tip"
)

plt.title("Total Bill vs Tip")

plt.show()
```



Interpretation

The graph helps visualize whether larger bills are generally associated with larger tips.

8.2 Bar Plot

Description

A bar plot is useful for comparing numerical values across different categories.

Seaborn can automatically calculate an aggregate statistic such as the mean for each category.

Use Case

We can compare the average total bill for different days.

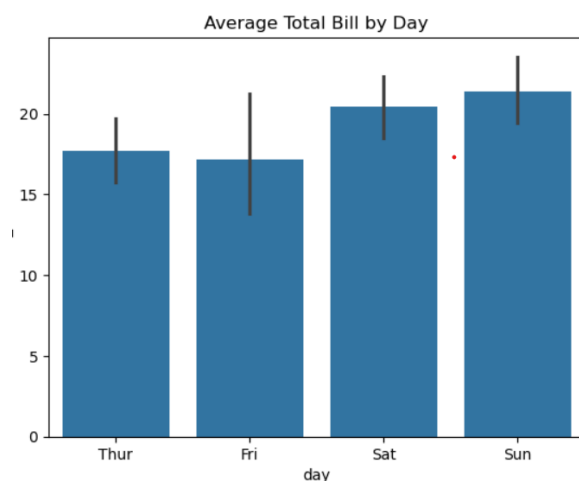
Code

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")

sns.barplot(
    data=data,
    x="day",
    y="total_bill"
)

plt.title("Average Total Bill by Day")
plt.show()
```



Interpretation

The graph compares the average bill amount across different days.

8.3 Histogram

Description

A histogram shows the distribution of numerical observations.

Use Case

It can be used to understand how restaurant bills are distributed.

Code

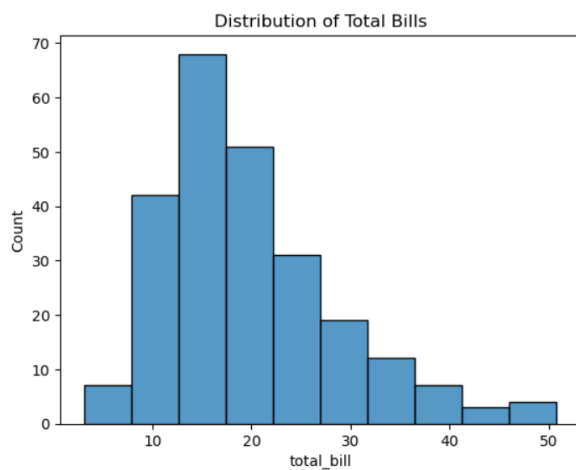
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")

sns.histplot(
    data=data,
    x="total_bill",
    bins=10
)

plt.title("Distribution of Total Bills")

plt.show()
```



Interpretation

The histogram shows the frequency of different total bill ranges.

8.4 Box Plot

Description

A box plot displays the distribution of numerical data and can help identify potential outliers.

Use Case

We can compare restaurant bill distributions across different days.

Code

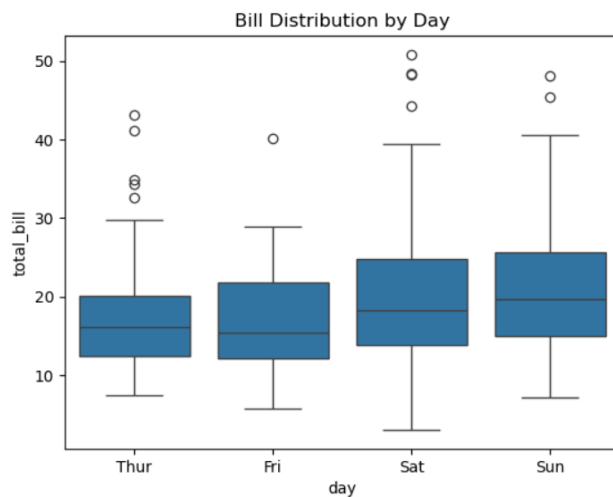
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")

sns.boxplot(
    data=data,
    x="day",
    y="total_bill"
)

plt.title("Bill Distribution by Day")

plt.show()
```



Interpretation

The plot allows us to compare the median, spread, and potential outliers between different days.

8.5 Violin Plot

Description

A violin plot combines information about the distribution and summary statistics of numerical data.

It provides more information about the shape of a distribution than a basic box plot.

Use Case

It can be used to compare the distribution of restaurant bills across different days.

Code

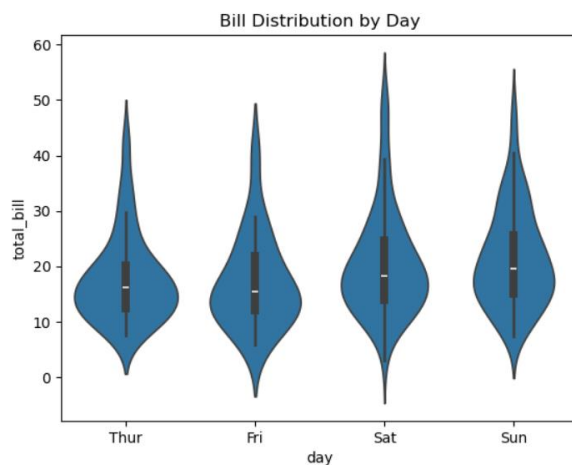
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")

sns.violinplot(
    data=data,
    x="day",
    y="total_bill"
)

plt.title("Bill Distribution by Day")

plt.show()
```



Interpretation

The width of each violin represents the density of observations at different values.

8.6 Heatmap

Description

A heatmap represents numerical values using variations in visual intensity.

It is commonly used to display a correlation matrix.

Use Case

Heatmaps can help identify relationships between numerical variables.

Code

```
import seaborn as sns
import matplotlib.pyplot as plt

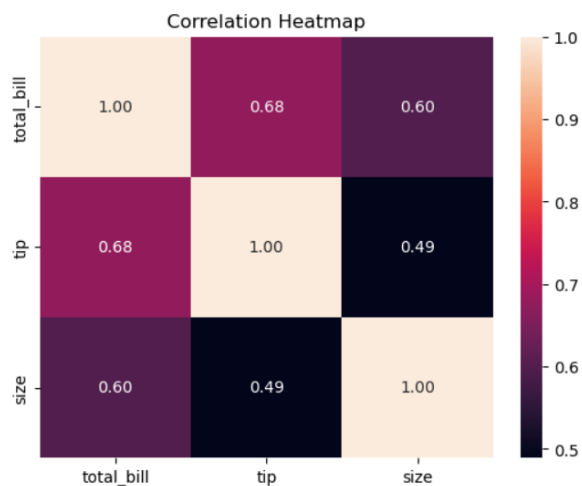
data = sns.load_dataset("tips")

correlation = data.select_dtypes(
    include="number"
).corr()

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```



Interpretation

Values closer to 1 indicate a strong positive correlation, values closer to -1 indicate a strong negative correlation, and values around 0 indicate a weak linear relationship.

Important: Correlation does not necessarily mean causation.

9. Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Main purpose	General-purpose visualization	Statistical and data-focused visualization
Ease of use	Moderate	Generally easier for statistical plots
Customization	Very high	High, with Matplotlib underneath
Default appearance	Basic	More polished statistical styling
Pandas integration	Good	Excellent
Statistical plots	Requires more manual work	Convenient built-in functions
Graph variety	Very wide	Wide, especially for statistical graphics
Interactivity	Limited compared with interactive-first libraries	Primarily static, uses Matplotlib
Learning curve	Moderate	Relatively beginner-friendly
Large datasets	Can work well, but rendering many points can be expensive	Similar limitations for dense plots
Best suited for	Detailed/customized plots	Exploratory and statistical data analysis

10. Strengths and Weaknesses Comparison

Matplotlib – Strengths

- Highly customizable.
- Supports many graph types.
- Provides detailed control over charts.
- Suitable for scientific and technical plots.
- Works well with NumPy and Pandas.
- Mature and widely used.

Matplotlib – Weaknesses

- Some visualizations require more code.
- Default charts can require additional styling.
- Complex figures can become verbose.
- Interactive visualization is not its primary strength.

Seaborn – Strengths

- Easy to create statistical graphics.
- Attractive default appearance.
- Excellent integration with Pandas DataFrames.
- Useful for exploratory data analysis.
- Provides convenient functions for distributions and categorical data.
- Built on top of Matplotlib.

Seaborn – Weaknesses

- Less low-level control than directly using Matplotlib.
- Mainly designed for statistical and data-focused visualization.
- Advanced customization may require Matplotlib knowledge.
- Not primarily an interactive dashboard library.

11. Interactivity

Matplotlib and Seaborn are primarily used for static analytical visualizations.

They can support interactive behavior in certain environments, but they are not primarily designed as browser-first interactive visualization libraries.

For applications requiring rich web-based interactivity, dashboards, hover interactions, and browser-based visual exploration, libraries such as **Plotly** or **Bokeh** may be more appropriate.

Therefore:

- **Matplotlib:** Excellent control and customization.
- **Seaborn:** Excellent statistical visualization.
- **Plotly/Bokeh:** Better suited for rich interactive visualizations.

12. Performance with Large Datasets

When visualizing very large datasets, performance can become an important consideration.

Rendering millions of individual points may make any plotting library slower and may produce an overcrowded visualization.

Recommended approaches

1. Aggregate the data before plotting.
2. Sample the data when appropriate.
3. Avoid unnecessary labels and annotations.

4. Use appropriate chart types.
5. Reduce the number of visual elements.
6. Filter data to the relevant range.
7. Consider specialized visualization tools for extremely large datasets.

For example, instead of plotting millions of individual transactions, the data can be aggregated by month and visualized using a line chart.

13. Choosing the Right Graph

Choosing the correct graph depends on the type of question being asked.

Analytical Goal	Recommended Graph
Show trends over time	Line Plot
Compare categories	Bar Chart
Show relationship between two variables	Scatter Plot
Show data distribution	Histogram
Compare distributions	Box Plot
Show distribution shape	Violin Plot
Show proportions	Pie Chart
Show correlations	Heatmap
Identify outliers	Box Plot
Compare multiple groups	Bar/Box/Violin Plot

Example

If the question is:

"How did sales change over six months?"

Use a **Line Plot**.

"What is the distribution of employee salaries?"

Use a **Histogram** or **Box Plot**.

14. Practical Mini Project – Sales Data Analysis

14.1 Objective

The objective of this mini project is to demonstrate how Matplotlib and Seaborn can be used together to analyze a simple sales dataset.

The dataset contains:

- Month
- Sales
- Profit

14.2 Creating the Dataset

```
import pandas as pd

data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 13500, 18000, 21000, 24000],
    "Profit": [1800, 2400, 2100, 3200, 3900, 4700]
}

df = pd.DataFrame(data)

print(df)
```

	Month	Sales	Profit
0	Jan	12000	1800
1	Feb	15000	2400
2	Mar	13500	2100
3	Apr	18000	3200
4	May	21000	3900
5	Jun	24000	4700

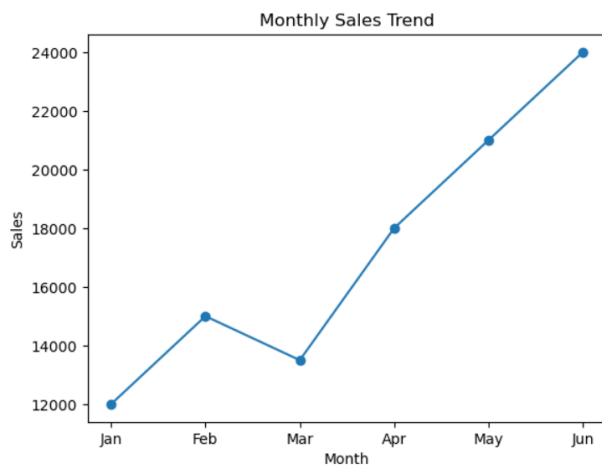
14.3 Visualizing Sales Trend Using Matplotlib

```
import matplotlib.pyplot as plt

plt.plot(
    df["Month"],
    df["Sales"],
    marker="o"
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
```

```
plt.show()
```



Interpretation

The line plot shows the sales trend from January to June.

The sales generally increase over the observed period, although there is a small decrease from February to March.

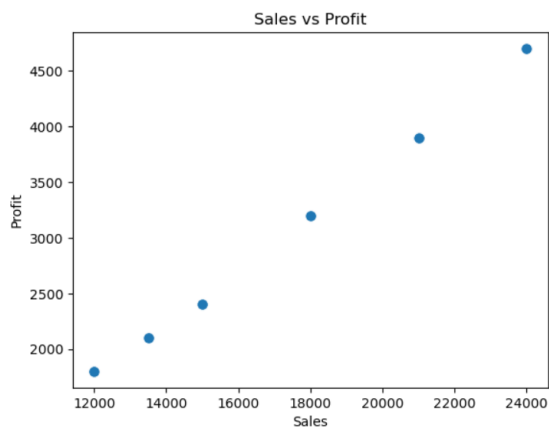
14.4 Visualizing Sales and Profit Using Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",
    s=70
)
```

```
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
```

```
plt.show()
```



Interpretation

The scatter plot helps examine the relationship between sales and profit.

The example data shows that higher sales values are generally associated with higher profit values.

However, this small example is only for demonstration and should not be treated as sufficient evidence for a real business conclusion.

14.5 Mini Project Findings

From the visualizations:

1. Sales generally increased from January to June.
2. Profit also increased as sales increased.
3. March showed a small decline in sales compared with February.
4. The scatter plot suggests a positive relationship between sales and profit in this example.
5. Visualization makes these patterns easier to identify than examining the raw table alone.

15. Data Visualization Best Practices

A good visualization should communicate information clearly and avoid unnecessary complexity.

15.1 Choose the Correct Chart

Always select a graph based on the analytical question.

For example:

- Trends → Line chart
- Comparison → Bar chart
- Relationship → Scatter plot
- Distribution → Histogram
- Outliers → Box plot
- Correlation → Heatmap

15.2 Use Meaningful Titles

Every graph should have a clear title that explains what the visualization represents.

15.3 Label Axes

Always label the x-axis and y-axis appropriately.

Include units when necessary.

For example:

Revenue (₹)
Temperature (°C)
Distance (km)

15.4 Avoid Unnecessary Clutter

Avoid excessive:

- Colors
- Gridlines
- Labels
- Decorations
- 3D effects

The main purpose of the visualization is to communicate information.

15.5 Make Text Readable

Use appropriate font sizes and ensure titles, axis labels, and legends can be read easily.

15.6 Be Careful With Scales

Misleading axis ranges can make differences appear larger or smaller than they really are.

15.7 Explain the Findings

Do not provide only the graph.

Explain what the graph tells the reader.

15.8 Remember Correlation Is Not Causation

If two variables appear related, it does not automatically mean that one causes the other.

16. When Should You Use Matplotlib or Seaborn?

Use Matplotlib when:

- You need detailed customization.
- You want precise control over the figure.
- You need specialized or complex layouts.
- You are creating scientific or technical plots.
- You want to control individual plot elements.

Use Seaborn when:

- You are performing Exploratory Data Analysis.
- You are working with Pandas DataFrames.
- You need statistical visualizations.
- You want attractive charts with less code.
- You want to compare distributions or categories.

Using Both Libraries Together

Matplotlib and Seaborn are not necessarily competitors.

They can be used together.

For example:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data=data,
    x="total_bill",
    y="tip"
)

plt.title("Total Bill vs Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")

plt.show()
```

17. Conclusion

Matplotlib and Seaborn are two important Python libraries for data visualization.

Matplotlib provides a flexible and highly customizable foundation for creating a wide variety of visualizations. It is suitable for scientific, technical, and general-purpose plotting.

Seaborn is built on top of Matplotlib and provides a higher-level interface that makes statistical and exploratory visualizations easier to create. It works particularly well with Pandas DataFrames.

The appropriate library and graph type depend on the analytical requirement.

For example:

- Line plots are useful for trends.
- Bar charts are useful for comparisons.
- Scatter plots are useful for relationships.
- Histograms are useful for distributions.
- Box plots are useful for distributions and outlier analysis.
- Violin plots are useful for detailed distribution comparisons.
- Heatmaps are useful for correlation analysis.

Understanding these visualization libraries enables beginners to transform raw data into meaningful visual insights and communicate analytical findings effectively.

18. Resources

The following official documentation was used as the primary reference for this guide.

Matplotlib

Matplotlib Official Documentation:

https://matplotlib.org/stable/users/explain/quick_start.html

The documentation explains Matplotlib's basic usage, figures, axes, plotting functions, customization, and visualization concepts.

Seaborn

Seaborn Official Documentation:

<https://seaborn.pydata.org/tutorial/introduction.html>

The documentation explains Seaborn's statistical visualization capabilities, datasets, plotting functions, and relationship with Matplotlib.

